EXP - 14

Exploring Pro-Code Extension of a Zoho Creator Application Using Zoho Catalyst / Custom Functions

Aim

To explore how a Zoho Creator application can be extended beyond low-code using custom functions, APIs and Zoho Catalyst.

Introduction

Low-code covers most requirements, but some needs (heavy computation, special libraries, complex integrations) call for real programming. Zoho offers a path from low-code to pro-code: Deluge custom functions inside Creator, and Zoho Catalyst, a serverless full-stack platform, for external logic written in languages such as Java, Node.js or Python.

Extension Options

1. Deluge Custom Functions

- Deluge is Zoho's scripting language used in workflows, buttons, schedules and validations.
- Custom functions let you write reusable logic once and call it from many places.
- Good for calculations, data transformations, calling REST APIs (invokeUrl), and sending mails.

2. Client-Side Customisation

- Client functions and Widgets built with HTML, CSS and JavaScript extend the user interface.
- Widgets can embed custom charts, maps or third-party libraries into pages and forms.

3. REST APIs

- Zoho Creator exposes REST APIs to create, read, update and delete records and to fetch reports.
- Authentication uses OAuth 2.0 (access and refresh tokens, scopes).
- External applications (web, mobile, other systems) can use these APIs.

4. Zoho Catalyst (Serverless Platform)

Catalyst provides back-end building blocks without managing servers:

- Functions: serverless code. Common types are Basic I/O, Advanced I/O (HTTP endpoints), Event, Cron and Browser Logic functions.
- Data Store: relational cloud database.
- File Store and Cache: storage and fast caching.
- Authentication: user sign-in services.
- Zia / AI services: OCR, text analysis and similar features.
- Web client hosting: host front-end apps.
- Languages: Java, Node.js and Python are supported for functions.

Integration Architecture

User -> Zoho Creator UI -> Deluge (invokeUrl) or Widget -> Catalyst Function (REST endpoint) -> processing / database / AI -> response -> Creator updates records or displays result.

Alternatively, Catalyst code can call the Creator REST API to push or fetch data.

Typical Implementation Steps

1. Identify a requirement beyond low-code (e.g., generate a complex PDF, run a scoring algorithm, call a custom ML model).
2. Create a Catalyst project and a function; write and test the code locally using the Catalyst CLI.
3. Deploy the function and note its endpoint URL.
4. Create a Connection in Zoho Creator (OAuth or API key) to authorise calls.

5. In Creator, use a Deluge script with invokeUrl to call the Catalyst endpoint, passing record data as JSON.
6. Parse the response and update records or show messages.
7. Add error handling, logging and security (restrict who can call the endpoint).
8. Test end to end and deploy through environments.

Example Use Cases

- Student skill scoring algorithm running as a Catalyst function and returning a level to Creator.
- RAG/LLM-based text summarisation service triggered from a Creator form.
- Advanced data validation or encryption.
- Bulk data processing too heavy for Deluge limits.

Benefits

- Overcomes low-code limits on logic, performance and libraries.
- Keeps rapid UI/workflow building in Creator while using full programming where needed.
- Serverless, so scaling and infrastructure are managed by the platform.
- Reusable services across multiple apps.

Challenges

- Needs developers with programming skills.
- More components to test, secure and monitor.
- Must manage authentication and API limits between systems.
- Debugging across two platforms.

Best Practices

- Keep functions small and single-purpose.
- Use JSON for data exchange and validate inputs.
- Store secrets in connections or configuration, not in code.
- Handle failures with retries and clear error messages.

- Version and document custom functions.

Conclusion

Combining Zoho Creator with custom functions and Zoho Catalyst delivers a hybrid development model: quick low-code delivery for most features and pro-code power for the hard parts.